

ECM2433

Mock Paper 2

The C Family

Closed Book

Additional Materials: None

Permitted Materials: No calculators permitted

Guidelines:

The duration of this paper is 2hr 00.

Answer ALL questions.

Write all answers in the answer booklet provided.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

This is a mock paper in the style of the May paper. Possible answers are skeleton answers.

Answers and marking notes are at the end of this document.

Question 1

(a) Complete `parse_positive` so it accepts only a whole positive decimal int from text. It must reject no digits, trailing junk, range errors, zero, and values too large for int. (5 marks)

```
#include <errno.h>
#include <limits.h>
#include <stdlib.h>

int parse_positive(const char *text, int *out) {
    char *end = NULL;
    errno = 0;
    long value = strtol(text, &end, 10);

    /* checks here */

    *out = (int)value;
    return 1;
}
```

(b) Complete the file fragment so it opens the named file, reports an open failure using `errno` and `strerror`, reads one line safely, and closes the file on the successful path. (5 marks)

```
#include <errno.h>
#include <stdio.h>
#include <string.h>

int print_first_line(const char *path) {
    FILE *fp = fopen(path, "r");
    char line[80];

    /* failure handling and read here */

    return 0;
}
```

(c) The helper `make_node` should be usable only inside `stack.c`, while `push` should be callable from other files. Fix the declarations and state which declaration belongs in the header. (4 marks)

```
/* stack.c */
Node *make_node(char value);
Stack push(Stack stack, char value);
```

(d) Complete `remove_first` so it removes the first linked-list node with the given key, updates the caller's head pointer when needed, frees the removed node, and returns 1 on success or 0 if no node matched. (7 marks)

```
#include <stdlib.h>

typedef struct Node {
    char key;
    struct Node *next;
} Node;

int remove_first(Node **head, char key) {
    /* complete this function */
}
```

(e) A loop reads characters with `fgetc` until EOF. Add the check that distinguishes a read error from a clean end-of-file, and explain why EOF alone is not enough. (4 marks)

```
int ch;
long count = 0;

while ((ch = fgetc(fp)) != EOF) {
    count++;
}

/* error check here */
```

(Total 25 marks)

Question 2

(a) Complete operator[] so assignment through grades[2] changes the stored element. Include a simple bounds check. (5 marks)

```
#include <stdexcept>

class Grades {
    int data[4] = {0, 0, 0, 0};

public:
    /* operator[] here */
};
```

(b) Complete the output operator for Date so std::cout << d prints day/month/year and can still be chained with another output. (5 marks)

```
#include <iostream>

class Date {
    int day;
    int month;
    int year;

public:
    Date(int d, int m, int y) : day(d), month(m), year(y) {}
    /* friend declaration here */
};

/* operator definition here */
```

(c) Write the out-of-class definitions for the Box template constructor and get method. Use the correct template headers and qualified names, and state where template definitions should normally live. (5 marks)

```
template<typename T>
class Box {
    T value;

public:
    Box(T value);
    T get() const;
};

/* definitions here */
```

(d) Trace the construction and destruction output for the automatic object in main, then explain the order. (5 marks)

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "B+"; }
    ~Base() { std::cout << "B-"; }
};

class Derived : public Base {
public:
    Derived() { std::cout << "D+"; }
    ~Derived() { std::cout << "D-"; }
};

int main() {
    Derived item;
}
```

(e) Write make_shape so it returns unique ownership of a Circle through the Shape interface. Explain why Shape's destructor must be virtual. (5 marks)

```
#include <memory>

class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius;

public:
    explicit Circle(double radius) : radius(radius) {}
    double area() const override { return 3.14159 * radius * radius; }
};

/* make_shape here */
```

(Total 25 marks)

Question 3

(a) Fix the lifetime error in `choose_label`. Explain what the lifetime annotation says and why it does not keep either input alive for longer. (5 marks)

```
fn choose_label(primary: &str, fallback: &str) -> &str {
    if primary.is_empty() {
        fallback
    } else {
        primary
    }
}
```

(b) Explain what the closure captures, why the closure variable must be mutable, and name the closure trait it needs in order to update `seen`. (4 marks)

```
let mut seen = 0;

let record = |word: &str| {
    if word.starts_with('a') {
        seen += 1;
    }
};

record("apple");
record("pear");
```

(c) Write an iterator pipeline that collects the index and value for every present positive reading into `Vec<(usize, i32)>` without using explicit indexing. (5 marks)

```
let readings = vec![Some(3), None, Some(-2), Some(8)];
/* collect [(0, 3), (3, 8)] */
```

(d) Complete `total_file` so it reads the file, parses each line as `i32`, adds the values, and propagates both I/O and parse errors with the question-mark operator. (6 marks)

```
use std::error::Error;
use std::fs;

fn total_file(path: &str) -> Result<i32, Box<dyn Error>> {
    /* complete */
}
```

(e) Rewrite `third_name` so it does not panic when the slice is too short. It should return `Option<&str>` borrowed from the input. (5 marks)

```
fn third_name(names: &[String]) -> &str {
    &names[2]
}
```

(Total 25 marks)

Question 4

(a) Complete `copy_slot` so it copies the array element at source index from into destination index to. Explain why the element width is needed. (6 marks)

```
#include <stddef.h>
#include <string.h>

void copy_slot(void *array, size_t from, size_t to, size_t width) {
    /* complete */
}
```

(b) Complete the lambda so it reads limit, updates hits, and can be called for every value in the vector. (4 marks)

```
#include <vector>

int limit = 10;
int hits = 0;
std::vector<int> values = {4, 12, 15, 7};

auto record = /* lambda here */;

for (int value : values) {
    record(value);
}
```

(c) Complete the loop so repeated product names accumulate totals in the HashMap instead of replacing earlier totals. (5 marks)

```
use std::collections::HashMap;

let rows = vec![
    (String::from("tea"), 2.00),
    (String::from("tea"), 1.50),
    (String::from("cake"), 3.25),
];

let mut totals: HashMap<String, f64> = HashMap::new();

for (name, price) in rows {
    /* update totals */
}
```

(d) Implement Iterator for Countdown so it yields current, current - 1, down to 1, and then stops. (5 marks)

```
struct Countdown {
    current: i32,
}

impl Iterator for Countdown {
    type Item = i32;

    fn next(&mut self) -> Option<Self::Item> {
        /* complete */
    }
}
```

(e) The function tries to calculate an array length using `sizeof` after the array is passed to a function. Explain the bug and rewrite the function signature and call so the length is handled correctly. (5 marks)

```
#include <stdio.h>

void print_count(int values[]) {
    size_t count = sizeof(values) / sizeof(values[0]);
    printf("%zu\n", count);
}

int main(void) {
    int values[5] = {1, 2, 3, 4, 5};
    print_count(values);
}
```

(Total 25 marks)

Answers and marking notes

These are skeleton answers. Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (5 marks): Complete `parse_positive` so it accepts only a whole positive decimal int from text. It must reject no digits, trailing junk, range errors, zero, and values too large for int.

Answer: Use `strtol`, then check whether no digits were consumed, whether trailing characters remain, whether `errno` reports range trouble, and whether the value is within the accepted int range.

```
if (end == text) {
    return 0;
}
if (*end != '\0') {
    return 0;
}
if (errno == ERANGE || value <= 0 || value > INT_MAX) {
    return 0;
}
```

(b) Question (5 marks): Complete the file fragment so it opens the named file, reports an open failure using `errno` and `strerror`, reads one line safely, and closes the file on the successful path.

Answer: Check `fopen` before using the stream. Use `strerror(errno)` only after the failed call. `fgets` reads at most `sizeof line - 1` characters and adds a terminator when it succeeds.

```
if (fp == NULL) {
    fprintf(stderr, "cannot open %s: %s\n", path, strerror(errno));
    return 1;
}

if (fgets(line, sizeof line, fp) == NULL) {
    if (ferror(fp)) {
        fprintf(stderr, "read error: %s\n", strerror(errno));
    }
    fclose(fp);
    return 1;
}

printf("%s", line);
fclose(fp);
return 0;
```

(c) Question (4 marks): The helper `make_node` should be usable only inside `stack.c`, while `push` should be callable from other files. Fix the declarations and state which declaration belongs in the header.

Answer: The helper should be static in `stack.c` so it has internal linkage. The public function should have a prototype in the header; `extern` is optional on a function prototype but may be written for clarity.

```
/* stack.c */
static Node *make_node(char value);
Stack push(Stack stack, char value);

/* stack.h */
Stack push(Stack stack, char value);
```

(d) Question (7 marks): Complete `remove_first` so it removes the first linked-list node with the given key, updates the caller's head pointer when needed, frees the removed node, and returns 1 on success or 0 if no node matched.

Answer: Use a pointer to the link that leads to the current node. This handles removal from the head and from the middle with the same assignment.

```
int remove_first(Node **head, char key) {
    Node **link = head;

    while (*link != NULL) {
        Node *current = *link;

        if (current->key == key) {
            *link = current->next;
            free(current);
            return 1;
        }

        link = &current->next;
    }

    return 0;
}
```

(e) Question (4 marks): A loop reads characters with `fgetc` until EOF. Add the check that distinguishes a read error from a clean end-of-file, and explain why EOF alone is not enough.

Answer: `fgetc` returns EOF both at end-of-file and on read error. After the loop, `ferror` tells you whether the stream saw an error.

```
    if (ferror(fp)) {
        return -1;
    }

    return count;
```

Question 2

(a) Question (5 marks): Complete `operator[]` so assignment through `grades[2]` changes the stored element. Include a simple bounds check.

Answer: The subscript operator must return a reference so the left-hand side assignment changes the stored element. A `const` overload could be added for read-only objects, but the assignment case needs `int&`.

```
int& operator[](unsigned int i) {
    if (i >= 4) {
        throw std::out_of_range("grade index");
    }
    return data[i];
}
```

(b) Question (5 marks): Complete the output operator for `Date` so `std::cout << d` prints day/month/year and can still be chained with another output.

Answer: `operator<<` is normally a non-member because the left operand is the stream. Returning the same stream by reference allows chaining such as `std::cout << d << '\n'`.

```
class Date {
    int day;
    int month;
    int year;

public:
    Date(int d, int m, int y) : day(d), month(m), year(y) {}
    friend std::ostream& operator<<(std::ostream& os, const Date& d);
};

std::ostream& operator<<(std::ostream& os, const Date& d) {
    os << d.day << "/" << d.month << "/" << d.year;
    return os;
}
```

(c) Question (5 marks): Write the out-of-class definitions for the `Box` template constructor and `get` method. Use the correct template headers and qualified names, and state where template definitions should normally live.

Answer: Each out-of-class template definition needs `template<typename T>`, and the class name must be `Box<T>`.

Template definitions normally live in the header so the compiler can instantiate them for the used types.

```
template<typename T>
Box<T>::Box(T value) : value(value) {}

template<typename T>
T Box<T>::get() const {
    return value;
}
```

(d) Question (5 marks): Trace the construction and destruction output for the automatic object in `main`, then explain the order.

Answer: The output is `B+D+D-B-`. Base subobjects are constructed before the derived part. Destruction happens in the reverse order: derived destructor first, then base destructor.

(e) Question (5 marks): Write `make_shape` so it returns unique ownership of a `Circle` through the `Shape` interface. Explain why `Shape`'s destructor must be virtual.

Answer: Return a `unique_ptr` to the base type, constructed with a derived `Circle`. The virtual destructor ensures deleting through a `Shape` pointer runs the derived destructor correctly.

```
std::unique_ptr<Shape> make_shape(double radius) {
    return std::make_unique<Circle>(radius);
}
```

Question 3

(a) Question (5 marks): Fix the lifetime error in `choose_label`. Explain what the lifetime annotation says and why it does not keep either input alive for longer.

Answer: The output may be either input, so one lifetime parameter ties the returned borrowed string slice to both input borrowed string slices. The annotation describes the relationship; it does not extend either string's real lifetime.

```
fn choose_label<'a>(primary: &'a str, fallback: &'a str) -> &'a str {
    if primary.is_empty() {
        fallback
    } else {
        primary
    }
}
```

(b) Question (4 marks): Explain what the closure captures, why the closure variable must be mutable, and name the closure trait it needs in order to update seen.

Answer: The closure mutably borrows seen because it increments it. Calling a closure that mutates captured state requires a mutable closure binding, and the closure implements FnMut.

```
let mut seen = 0;

let mut record = |word: &str| {
    if word.starts_with('a') {
        seen += 1;
    }
};

record("apple");
record("pear");
```

(c) Question (5 marks): Write an iterator pipeline that collects the index and value for every present positive reading into Vec<(usize, i32)> without using explicit indexing.

Answer: enumerate supplies the index, and filter_map can discard None and non-positive values while producing the kept tuple.

```
let positives: Vec<(usize, i32)> = readings
    .iter()
    .enumerate()
    .filter_map(|(index, reading)| match reading {
        Some(value) if *value > 0 => Some((index, *value)),
        _ => None,
    })
    .collect();
```

(d) Question (6 marks): Complete total_file so it reads the file, parses each line as i32, adds the values, and propagates both I/O and parse errors with the question-mark operator.

Answer: The function returns Result so the question-mark operator can propagate either the file-read error or the parse error. Box<dyn Error> can hold either error type.

```
fn total_file(path: &str) -> Result<i32, Box<dyn Error>> {
    let text = fs::read_to_string(path)?;
    let mut total = 0;

    for line in text.lines() {
        total += line.trim().parse::<i32>()?;
    }

    Ok(total)
}
```

(e) Question (5 marks): Rewrite third_name so it does not panic when the slice is too short. It should return Option<&str> borrowed from the input.

Answer: Indexing with names[2] panics if there is no third element. get returns Option, and map converts the borrowed String to a borrowed string slice.

```
fn third_name(names: &[String]) -> Option<&str> {
    names.get(2).map(|name| name.as_str())
}
```

Question 4

(a) Question (6 marks): Complete copy_slot so it copies the array element at source index from into destination index to. Explain why the element width is needed.

Answer: The function does not know the real element type, so it must use the supplied width to calculate byte offsets. Convert the base pointer to char * so pointer arithmetic moves one byte at a time.

```
void copy_slot(void *array, size_t from, size_t to, size_t width) {
    char *bytes = array;

    memmove(bytes + to * width, bytes + from * width, width);
}
```

(b) Question (4 marks): Complete the lambda so it reads limit, updates hits, and can be called for every value in the vector.

Answer: Capture limit by value because the lambda only reads it. Capture hits by reference because the lambda must update the original counter.

```
auto record = [limit, &hits](int value) {
    if (value > limit) {
        hits++;
    }
};
```

(c) Question (5 marks): Complete the loop so repeated product names accumulate totals in the HashMap instead of replacing earlier totals.

Answer: entry finds the existing total or inserts 0.0. The returned mutable reference is dereferenced so the price is added to the stored total.

```
for (name, price) in rows {
    let total = totals.entry(name).or_insert(0.0);
    *total += price;
}
```

(d) Question (5 marks): Implement Iterator for Countdown so it yields current, current - 1, down to 1, and then stops.

Answer: next returns Some(item) while the iterator can yield another value, then None when iteration is finished. The struct stores the current state.

```
fn next(&mut self) -> Option<Self::Item> {
    if self.current <= 0 {
        None
    } else {
        let value = self.current;
        self.current -= 1;
        Some(value)
    }
}
```

(e) Question (5 marks): The function tries to calculate an array length using sizeof after the array is passed to a function. Explain the bug and rewrite the function signature and call so the length is handled correctly.

Answer: In a function parameter, int values[] is adjusted to int *values. sizeof(values) is therefore the size of a pointer, not the original array. Pass the length separately.

```
void print_count(const int values[], size_t count) {
    printf("%zu\n", count);
}

int main(void) {
    int values[5] = {1, 2, 3, 4, 5};
    print_count(values, sizeof values / sizeof values[0]);
}
```